



Solo Challenge.

Seven days of learning. One app. Built alone. Today you prove you can take an idea and ship it — the skill that matters for the group project, for Demo Day, and for every job after.

PROGRAM

Python Internship

LED BY

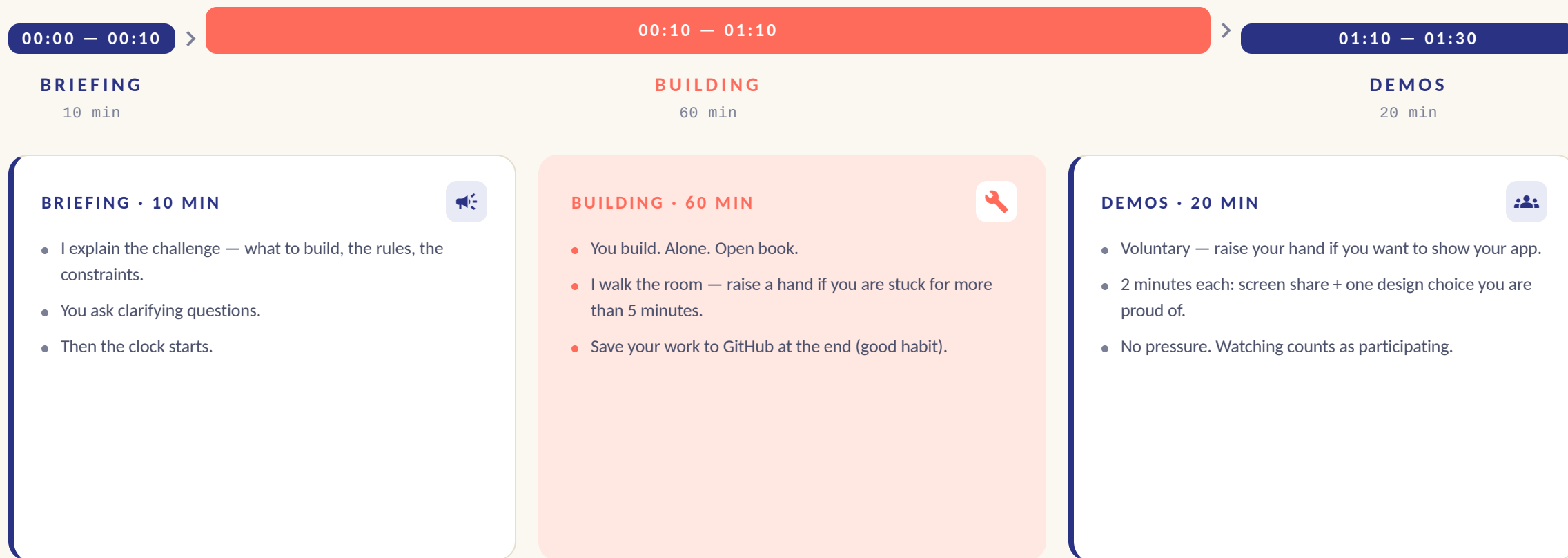
Zlaark

SESSION

8 of 24

Ten minutes of briefing. Sixty minutes of building. Twenty minutes of demos.

No new concepts today. You already have everything you need from Days 1–7. Today is about using it.



Build a personal expense tracker.

The requirements are below. The UI structure, the layout, the data flow — those are YOUR decisions. Brainstorm them before you write a single line of code.

REQUIREMENTS • THE APP MUST

1 Let users add an expense

Each expense has: amount (number), category (text or dropdown), date (date), and an optional note (text). Users must be able to submit a new expense through the app.

2 Store expenses so they persist between runs

When the user closes the app and reopens it, the expenses should still be there. You decide how to store them — CSV file, JSON file, or SQLite database. (Hint: CSV is simplest. You learned it on Day 6.)

3 Show the total amount spent

A single, prominent number showing the sum of all expenses. You decide where it goes and how it looks.

4 Visualize spending by category

A chart showing how spending breaks down across categories (Food, Transport, Books, etc.). You choose the chart type — bar, pie, or something else. (Hint: bar charts are easiest to read.)

5 Show a list of recent expenses

A table or list of the expenses the user has added. You decide how many to show and whether it is sortable/filterable.

YOUR DECISIONS • BRAINSTORM FIRST

- ☐ Layout — sidebar + main? columns? tabs? single page?
- ☐ Where does the “add expense” form go?
- ☐ Where does the total go? (st.metric? heading?)
- ☐ Where does the chart go?
- ☐ Where does the table go?
- ☐ How do you load the data on startup?
- ☐ How do you save a new expense? (append to CSV? rewrite the whole file?)
- ☐ What categories will you offer? (Free text? A fixed list?)
- ☐ Will you use st.form or individual widgets?
- ☐ What happens if the CSV does not exist yet? (First run.)

BONUS (IF TIME ALLOWS)

Filter expenses by category or date range. Add a “delete expense” feature. Show spending over time as a line chart. These are NOT required — finish the core 5 first.

— 4 LAYOUTS • PICK ONE OR DESIGN YOUR OWN

Four ways to lay out *the same app*.

The requirements are fixed. The layout is not. Here are four common patterns — each works, each is used in real apps. Pick one, mix two, or design your own. The next 4 slides show what each looks like.

1  Sidebar + Main

2  Top Form + Dashboard

3  Tabs (Add / View / Chart)

4  Two-Column

Layout 1 — Sidebar + Main. The classic dashboard.

Controls in the left sidebar, output in the main area. The pattern from Day 5's sidebar deep-dive. Most common in real analytics dashboards.

localhost:8501 — Expense Tracker

+ ADD EXPENSE

Amount (₹)

450

Category

Food

Date

2026-08-04

Note

Lunch...

Add Expense

My Expense Tracker

TOTAL SPENT

₹ 12,450

Spending by Category

Recent Expenses

DATE	CATEGORY	NOTE	AMOUNT
04 Aug	Food	Lunch	₹ 450
03 Aug	Transport	Auto	₹ 120
03 Aug	Books	Python	₹ 680

HOW IT WORKS

- 1 Sidebar = the form.** `with st.sidebar:` wraps all the input widgets. Users add expenses from the left.
- 2 Main area = output.** `st.metric` for total, `st.bar_chart` for breakdown, `st.dataframe` for the table.
- 3 Controls stay put.** The sidebar doesn't scroll with the main area. Good when the form is used repeatedly.

BEST FOR: Apps where users add data frequently but view output less often. The classic analytics dashboard pattern.

DAY 08 • SOLO CHALLENGE

05 / 15

Layout 2 — Top form + dashboard below. No sidebar.

The form sits at the top of the main area, with metrics + chart + table stacked below. Simpler than a sidebar — everything on one page, top to bottom.

localhost:8501 — Expense Tracker

My Expense Tracker

+ ADD EXPENSE

Amount (₹)

450

Category

Food

Date

04 Aug

Note

Lunch with team

Add

TOTAL SPENT

₹ 12,450

THIS MONTH

₹ 3,200

Spending by Category

Recent Expenses

DATE	CATEGORY	NOTE	AMOUNT
04 Aug	Food	Lunch	₹ 450
03 Aug	Transport	Auto	₹ 120

HOW IT WORKS

- 1

Form at the top. A `st.form` or a card with widgets, laid out horizontally using `st.columns`. Submit button on the right.
- 2

Dashboard below. Metrics in a row (`st.columns(2)`), then the chart, then the table — all stacked vertically.
- 3

No sidebar. Everything is in the main area. Simpler to build, simpler to scroll.

BEST FOR: Apps where the form is used occasionally and the dashboard is the focus. Simpler than Layout 1 — good for beginners.

Layout 3 — Tabs. Separate views for Add, View, Chart.

Three tabs at the top. Users switch between “Add Expense”, “View Expenses”, and “Charts”. Each tab is a different screen — less clutter, more navigation.

The screenshot shows a web browser window titled 'localhost:8501 - Expense Tracker'. The application is titled 'My Expense Tracker' and has three tabs: 'Add Expense' (active), 'View Expenses', and 'Charts'. Below the tabs is a form to 'ADD A NEW EXPENSE' with fields for Amount (₹) (450), Category (Food), Date (04 Aug), and Note (Lunch...). An 'Add' button is next to the note field. Below the form is a summary section 'TOTAL SPENT (ALL TIME)' showing ₹ 12,450. At the bottom, there are two sections: 'By Category' with a horizontal bar chart showing four red bars of decreasing length, and 'Recent' with a table of recent expenses.

Recent	
Food	₹ 450
Transport	₹ 120

HOW IT WORKS

- 1 Three tabs.** `tab1, tab2, tab3 = st.tabs(["Add", "View", "Charts"])`. Each tab is a context manager.
- 2 Add tab = the form.** Users add expenses here. Only the form is visible on this screen.
- 3 View tab = the table.** Full `st.dataframe` with all expenses. Sortable, scrollable.
- 4 Charts tab = visualizations.** Bar chart, maybe a line chart over time. The analytics view.

BEST FOR: Apps with distinct workflows. Users who want to focus on one task at a time. Feels more like a “real app”.

Layout 4 — Two-column. Form left, dashboard right.

Like the sidebar layout, but both panels are full-height columns in the main area. Uses `st.columns(2)`. Form and dashboard visible at once.

localhost:8501 — Expense Tracker

My Expense Tracker

+ ADD EXPENSE

Amount (₹)

450

Category

Food

Date

04 Aug

Note

Lunch...

Add Expense

TOTAL SPENT

₹ 12,450

By Category

Recent

DATE	CAT.	AMT
04 Aug	Food	₹ 450
03 Aug	Transport	₹ 120
03 Aug	Books	₹ 680

HOW IT WORKS

- Two columns.** `col1, col2 = st.columns(2)`. Left column = form, right column = dashboard.
- Both visible at once.** Unlike tabs, the form and the output are on screen together. Add an expense, see the chart update immediately.
- Use ratios for balance.** `st.columns([1, 2])` makes the right column twice as wide — more room for the chart and table.

BEST FOR: Wide screens. Apps where seeing the form and the result together matters. Feels like a live editor.

Open book. No teammates. Sixty minutes.

The rules are simple. The point is to prove YOU can build it — not that you can copy it.

ALLOWED



- ✓ Your Day 1–7 notes, cheatsheets, and code files
- ✓ The pandas, Streamlit, and Matplotlib official documentation
- ✓ Google / Stack Overflow for specific errors
- ✓ Your own past code (Day 4–7 apps you built)
- ✓ Asking the instructor questions (raise a hand)

NOT ALLOWED



- ✗ Copying code from a classmate
- ✗ Asking a classmate for their solution
- ✗ AI code generators (Copilot, ChatGPT, etc.) — today is about YOUR brain
- ✗ Asking someone else to debug for you
- ✗ Starting before the briefing ends

THE SPIRIT

This is not a test with a passing grade. It is a checkpoint. If you get stuck, that is data — it tells us what to fix before the group project. Honesty matters more than completion.

Everything you need. You already learned it.

Four days of Streamlit. One day of data. One day of charts. Here is the whole toolkit in one place.

DAY 4 MEET STREAMLIT

- `st.title`, `st.write`, `st.text_input`, `st.button`
- The run-loop: every click re-runs the script
- `st.balloons()` for celebration

DAY 5 WIDGETS & LAYOUT

- `st.columns`, `st.sidebar`, `st.tabs`, `st.expander`
- `st.session_state` for persistence across reruns
- `st.form` for batch input (submit button)
- Widget keys — give every widget a key

DAY 6 WORKING WITH DATA

- `pd.read_csv`, `df.head`, `df.info`, `df.describe`
- `df[mask]` for filtering, `df.sort_values`
- `df.loc` vs `df.iloc`, `df["new"] =` for new columns
- `df.groupby("col")["col2"].mean()`

DAY 7 CHARTS & DASHBOARDS

- `st.line_chart`, `st.bar_chart`, `st.area_chart`
- Matplotlib: `fig, ax = plt.subplots()` + `st.pyplot(fig)`
- `st.metric` for KPI numbers
- `st.dataframe` for interactive tables

FILES YOU MIGHT NEED

- `expenses.csv` — where you store the data
- `app.py` — your Streamlit app
- Use pandas to load/append/save the CSV

THE PATTERN

Load data with pandas. **Show** it with `st.dataframe`.
Summarize it with `st.metric`. **Visualize** it with `st.bar_chart`.
Add new data with `st.form`. **Store** it back to CSV. That is the whole app.

A suggested breakdown. Ignore it if you have your own rhythm.

The #1 reason students fail this challenge is not lack of skill — it is spending 40 minutes on the chart and 0 minutes on the form. Scope discipline.

1

00:00–00:05 BRAINSTORM

- Answer the checklist from slide 3. On paper. Before you open VS Code.
- Decide: layout, storage, chart type, categories. 5 minutes.
- Do NOT start coding yet. Plan first.

2

00:05–00:20 SCAFFOLD

- Create `app.py`. Get the basic structure running: `st.title`, load CSV (or create it if missing).
- Add the form (`st.form` or individual widgets) — but do not wire up the save logic yet.
- Run it. See something on screen. 15 minutes.

3

00:20–00:45 CORE FEATURES

- Wire up the save: form submit → append to CSV → reload data.
- Add the total (`st.metric` or `st.write`).
- Add the chart (`groupby` + `st.bar_chart`).
- Add the table (`st.dataframe`).
- **25 minutes.** This is where the app becomes real.

4

00:45–01:00 POLISH + TEST

- Test: add 3-4 expenses, restart the app, confirm they persisted.
- Polish: titles, labels, spacing. Handle empty-state (what if no expenses yet?).
- If time: bonus features (filter, delete, line chart).
- **15 minutes.** Save to GitHub if you have time.

SCOPE

You will get stuck. That is normal. Here is what to do.

Do not stare at the screen for 10 minutes hoping the code fixes itself. Follow these three steps, in order.

1 STEP 1 READ THE ERROR

🕒 2 MIN

- The error message is not noise — it is the computer telling you exactly what is wrong.
- Read the file name and line number — that is where to look.
- **2 minutes.** If still stuck, go to Step 2.
- Read the **LAST line** first (that is where the actual error is).
- Google the exact error message. Stack Overflow has seen it before.

2 STEP 2 PRINT AND ISOLATE

🕒 5 MIN

- Add `print()` statements around the failing line. What values do the variables actually have?
- Try the failing line in a fresh Python file by itself. Does it work there?
- Comment out half the code. Does the error go away? You narrowed it down.
- **5 minutes.** If still stuck, go to Step 3.

3 STEP 3 RAISE YOUR HAND

🕒 NO LIMIT

- That is what I am here for. I will not give you the answer — I will help you find it.
- I would rather answer 30 questions today than watch you give up.
- There is no shame in asking. There **IS** shame in sitting stuck for 20 minutes.
- **No time limit** on this step. Ask.

COMMON STUCK POINTS (1) CSV not loading: check the path and encoding. (2) Form not submitting: did you use `st.form_submit_button` ? (3) Chart empty: did you `groupby` before charting? (4) Data not persisting: are you appending or overwriting?

Show your work. Or just watch. Both are fine.

Voluntary demos at the end. If you want to show your app, raise your hand. If you prefer to watch, that is okay too.

HOW IT WORKS

6 Rules

- **Voluntary** — raise your hand if you want to demo.
- **2 minutes each:** 30-second screen share + 90-second explanation.
- **Screen share:** open your app in the browser, walk through adding one expense.
- **Explanation:** tell us ONE design choice you are proud of (layout, chart type, storage, etc.).
- **No judgment on completion.** A working form + table is a win. A polished app with charts is a big win.
- If we run out of time, priority goes to students who have not demoed before.

WHAT TO SHOW

Checklist

- ✓ Open the app in your browser (`localhost:8501`)
- ✓ Add one expense live (show the form works)
- ✓ Show it appears in the table
- ✓ Show the total updated
- ✓ Show the chart (if you built one)
- ✓ Restart the app — show the data persisted (if it did!)
- ✓ Tell us one thing you would improve with more time

IF YOUR APP BROKE Demo it anyway. Showing a broken app and explaining **WHY** it broke is more valuable than hiding it. We learn from failures, not just successes.

This is not a test. It is a mirror.

The challenge shows you — and me — where you actually are. Not to grade you, but to know what to fix before the group project starts on Day 15.

WHAT THIS TESTS

5 Skills

- ✓ Can you take a requirement and design a UI for it? (Day 5 layout)
- ✓ Can you load and save data with pandas? (Day 6 data)
- ✓ Can you visualize that data with a chart? (Day 7 charts)
- ✓ Can you wire up a form that actually persists? (Day 5 session_state + Day 6 CSV)
- ✓ Can you debug when something breaks? (Every day)

WHAT COMES NEXT

Phase 3 Begins



Day 9 — Saving Data

JSON, CSV, SQLite. The storage patterns you touched today, formalized.



Day 10 — Working With Text

Regex, PDFs, Word docs. Pulling text out of messy sources.



Day 11 — Calling Real APIs

Live data from the internet. Weather, currency, news.

THE GATE If you struggled today, that is okay. Phase 3 fills the gaps. If you flew through it, Phase 3 goes deeper. Either way, you are ready.

TOMORROW We start Phase 3. Data storage, text processing, and live APIs. The skills that make the group-project ideas from Day 6 actually possible.

• CLOSING • 5 MIN

You built something real. Today.

An app that stores data, visualizes it, and persists between runs. Seven days ago, you installed Python. Today you shipped an app. That is the leap.

BEFORE YOU LEAVE

CHECKLIST

- ✓ Save your `app.py` to your `day8-practice` GitHub repo
- ✓ Commit with a message: `"Day 8 solo challenge — expense tracker"`
- ✓ Take a screenshot of your app — you will want it

BETWEEN SESSIONS

GET UNSTUCK

- ✓ **Email • `hello@zlaark.com`**
If your app broke and you want to know why, send the code + error.
- ✓ **Web • `www.zlaark.com`**
Program updates and resources.
- ✓ **In-session • raise a hand**
Tomorrow we start Phase 3. Come with questions.

WHAT'S NEXT • PHASE 3

DAYS 9-14

- ✓ **Day 9 — Saving Data.** JSON, CSV, SQLite. Storage formalized.
- ✓ **Days 10-11 — Text + APIs.** Regex, PDFs, live data.
- ✓ **Days 12-14 — A first taste of AI.** TF-IDF, similarity matching.